Singable Lyrics Translation with IPA-Augmented LLM Agents

Anonymous ACL submission

Abstract

Translating song lyrics while preserving singability requires satisfying musical constraints, including syllable counts, rhyme schemes, and syllable patterns, that standard machine translation ignores. We present a framework that exposes IPA-based phonetic analysis as a suite of composable Agent Skills, each defined by a natural-language description plus deterministic verification scripts that the orchestrating LLM invokes on demand. The agent extracts constraints from source lyrics, then translates through a multi-phase process, calling skills to iteratively verify and refine each constraint. We formalize the three musical constraints, describe the IPA skill suite and agent architecture, and propose three evaluation metrics: Syllable Error Rate (SER), Syllable Count Relative Error (SCRE), and Adjusted Rand Index (ARI). A phase ablation study confirms that iterative tool-verified refinement dramatically improves constraint satisfaction and preserves rhyme structure substantially better than a supervised baseline, while requiring no task-specific training, no parallel corpora, and generalizing to any of the 100+ languages supported by eSpeak-NG.

1 Introduction

A translated song lyric must conform to the musical structure of the original so that it remains *singable* to the same melody (Low, 2005). This requires satisfying hard structural constraints: syllable counts must match melodic phrases, rhyme schemes should be preserved, and word-level syllable distributions should mirror the original phrasing. Despite theoretical grounding (Low, 2005; Franzon, 2008), computational approaches remain scarce. Neural MT systems have no mechanism for phonetic constraints, and constrained decoding methods (Hokamp and Liu, 2017; Post and Vilar, 2018) address lexical but not phonetic constraints. A fundamental obstacle is that *counting syllables, detecting rhyme, and analyzing phonetic patterns are not tasks that LLMs perform reliably through text generation alone* (Gao et al., 2023).

We address this by exposing IPA-based phonetic analysis as a suite of Agent Skills, where each capability is packaged as a directory of natural-language instructions plus deterministic scripts that the orchestrating LLM discovers and invokes during translation (Yao et al., 2023). Unlike supervised approaches, the framework requires no parallel lyric corpora, supports any language pair via eSpeak-NG, and scales with LLM capability. Our framework is open-source for reproducibility and practical use.[1]

Our contributions are:

1. A formalization of three musical constraints (syllable counts, rhyme schemes, and syllable patterns) and IPA-based methods for their extraction and verification (§3).

2. An IPA-augmented Skill suite and multi-phase orchestration architecture for constraint-preserving lyrics translation (§4).

3. Three automatic evaluation metrics (SER, SCRE, and ARI) designed to measure structural constraint preservation in singable translation (§5).

To our knowledge, this is the first framework to package IPA-based phonetic analysis as Agent Skills for singable lyrics translation.

2 Related Work

2.1 Song Translation

Theoretical accounts treat song translation as balancing singability, sense, naturalness, rhythm, and rhyme (Low, 2005; Franzon, 2008; Low, 2022). Computationally, prior work has formalized singable lyric translation as constrained

[1] Code and Skills available at `https://github.com/anonymous` (anonymized for review).

sequence-to-sequence learning (Ou et al., 2023a), addressed tonal-language constraints (Guo et al., 2022), applied constrained decoding to neural poetry (Ghazvininejad et al., 2018), modeled musical features during generation (Ye et al., 2024), and used curriculum reinforcement learning (Ren et al., 2025). Related directions include melody-constrained lyric editing (Zhao et al., 2025), structural constraints in melody-to-lyric generation (Ou et al., 2023b), multilingual datasets and evaluation (Cho et al., 2025; Kim et al., 2023, 2024), scansion-based and prosody-aware lyric generation (Chen and Teufel, 2024; Liang et al., 2024; Yu et al., 2025), and multimodal singability (Yoo et al., 2025). These approaches rely on specialized fine-tuning, constrained decoding, or reinforcement learning; our work instead augments a general-purpose LLM with IPA verification skills, requiring no task-specific training. We position BLT Skills against three reference points: structurally constrained singable lyric translators that learn constraint satisfaction from parallel data (Ou et al., 2023a; Guo et al., 2022; Ren et al., 2025); controlled-poetry and prosody-aware generators that handle meter and rhyme inside the model (Ghazvininejad et al., 2018; Chen and Teufel, 2024; Liang et al., 2024); and benchmark-oriented evaluation efforts (Kim et al., 2023; Cho et al., 2025). Our scope is narrower and more pragmatic: a training-free skill suite whose contribution is the *external* IPA-verification interface and its multi-phase use, rather than a new generation model or a new dataset. We therefore do not retrain or evaluate against most of these systems but compare directly to the supervised constraint-token baseline that is closest in task setting (Ou et al., 2023a; §6).

2.2 Constrained Text Generation

Lexically constrained decoding has been studied via Grid Beam Search and Dynamic Beam Allocation (Hokamp and Liu, 2017; Post and Vilar, 2018), while computational poetry has used finite-state machinery and neural models for meter and rhyme (Ghazvininejad et al., 2016; Hopkins and Kiela, 2017; Lau et al., 2018). These methods enforce constraints at the decoding level or learn them implicitly; ours lets the LLM generate freely and verifies via external phonetic skills.

2.3 Tool-Augmented Language Models

Prior work has shown that LLMs can be trained or prompted to invoke external tools (Schick et al., 2023), interleave reasoning with tool calls (Yao et al., 2023), and offload computation to program execution (Gao et al., 2023), the latter directly relevant here since syllable counting is unreliable in text generation but deterministic via IPA tools. Chain-of-thought prompting (Wei et al., 2022) provides complementary structured reasoning that we combine with skill-decomposed tool use. More recently, frontier LLM platforms have introduced *Agent Skills*: capabilities packaged as a directory containing a natural-language description (`SKILL.md`) plus optional helper scripts that the orchestrating model loads on demand; we adopt this design to factor phonetic analysis into independent, reusable skills. In machine translation, Jiao et al. (2023) showed LLMs achieve strong translation quality, but did not consider musical or structural constraints.

2.4 Phonetic Resources

Our skill suite builds on Phonemizer (Bernard and Titeux, 2021) for text-to-IPA conversion and PanPhon (Mortensen et al., 2016) for phonetic distance computation; both are described in §4. Concurrently, Chen et al. (2025) explored using IPA as an intermediary representation for LLM-based translation of spoken-only languages; while their focus is on low-resource language preservation rather than musical constraints, both approaches share the insight that IPA provides a language-agnostic phonetic interface for LLMs.

3 Musical Constraints in Lyrics Translation

A singable translation must satisfy three structural constraints, each arising from a different aspect of the relationship between lyrics and melody.

3.1 Syllable Count Constraint

In sung performance, each syllable maps to one or more musical notes. When a melody assigns three notes to a phrase, the lyric must contain exactly three syllables; fewer leave notes unmatched, while extra syllables force cramming that disrupts the rhythmic feel. This makes syllable count the most rigid of the three constraints: rhyme and phrasing mismatches may be tolerable in some styles, but syllable count mismatches are almost always audible. Formally, given source lyrics $L_s = (l_1, \ldots, l_n)$ in language λ_s, the constraint requires that translated lyrics $L_t = (l'_1, \ldots, l'_n)$ in target language λ_t satisfy $\text{syll}(l'_i, \lambda_t) = \text{syll}(l_i, \lambda_s)$ for all lines i.

Syllable counting is language-dependent. For Chinese, each character constitutes exactly one syllable, so counting is trivial. For other languages, we convert text to IPA via Phonemizer (Bernard and Titeux, 2021) and count *vowel nuclei*, the vocalic centers of syllables:

$$\text{syll}(l_i, \lambda_s) = \left|\text{nuclei}\big(\text{IPA}(l_i, \lambda_s)\big)\right| \qquad (1)$$

where nuclei$(\cdot)$ matches a predefined set of IPA vowel symbols and diphthongs (e.g., a, i, u, ə, aɪ, eɪ, oʊ), with adjacent vowels forming diphthongs counted as single nuclei. For example, "*Let it go*" yields IPA /lɛt ɪt gəʊ/ with three vowel nuclei [ɛ, ɪ, əʊ], giving a count of 3. The resulting sequence $\mathbf{s} = (s_1, \ldots, s_n)$ serves as the hard constraint for translation.

3.2 Rhyme Scheme Constraint

Rhyme provides structural cohesion in lyrics, creating sonic expectations that are satisfying when met and jarring when violated. A song with an $AABB$ scheme feels fundamentally different from one with $ABAB$; preserving the rhyme scheme ensures the translation maintains the same pattern of sonic echoes as the original. Formally, a rhyme scheme is a labeling $R = (r_1, \ldots, r_n)$ where $r_i \in \{A, B, C, \ldots\}$ assigns each line to a rhyme class, with $r_i = r_j$ if and only if lines i and j rhyme. The constraint requires that the translation's scheme R_t match the source scheme R_s.

We detect rhyme by comparing phonetic endings rather than orthographic forms. For each line, we extract the *rhyme ending*: the IPA substring from the last vowel nucleus to the end of the transcription. For Chinese, we extract the pinyin final of the last character instead. The scheme is constructed by labeling the first line A, and assigning each subsequent line the label of any earlier line it rhymes with, or a new label otherwise. Consider the following example:

> *The stars shine bright with silver* ***light*** → /laɪt/ → ending: /aɪt/
> *And covers all in purest* ***white*** → /waɪt/ → ending: /aɪt/
> *I stand and watch the world* ***below*** → /bɪləʊ/ → ending: /əʊ/
> *As gentle winds begin to* ***blow*** → /bləʊ/ → ending: /əʊ/

Lines 1–2 share ending /aɪt/ (class A) and lines 3–4 share /əʊ/ (class B), yielding $AABB$. This IPA-based approach correctly identifies rhymes that orthographic comparison would miss (e.g., "weight" and "late") and rejects false rhymes based on spelling alone (e.g., "cough" and "through").

3.3 Syllable Pattern Constraint

Even when two lines share the same total syllable count, different word-level distributions create different rhythmic feels. Consider two eight-syllable lines: "un-for-get-ta-ble mel-o-dy" with pattern $[5, 3]$ versus "I can hear the mu-sic play" with pattern $[1, 1, 1, 1, 2, 2]$. The first has two long words producing a flowing feel, while the second has many short words producing a staccato rhythm. Word boundaries interact with note groupings and breaths in sung performance, so preserving the syllable pattern ensures the translation fits the melody at the phrasing level, not just the syllable level. Formally, the syllable pattern of a line l_i is a vector $\mathbf{p}_i = (p_{i,1}, \ldots, p_{i,k_i})$ recording the syllable count of each word. The constraint asks that the translation's pattern $\mathbf{p}'_i$ be similar to $\mathbf{p}_i$ while maintaining the same total: $\sum_j p'_{i,j} = \sum_j p_{i,j} = s_i$.

To extract patterns, we apply language-specific word segmentation (space-based tokenization for English and other space-delimited languages, and a neural tokenizer for Chinese), then compute each word's syllable count via Equation 1. We measure pattern similarity using a normalized alignment score. Given patterns of potentially different lengths, we pad the shorter one with zeros and compute:

$$\text{sim}(\mathbf{p}, \mathbf{p}') = 1 - \frac{\sum_{j=1}^{m} |p_j - p'_j|}{\sum_{j=1}^{m} \max(p_j, p'_j)} \qquad (2)$$

where $m = \max(|\mathbf{p}|, |\mathbf{p}'|)$. A score of 1.0 indicates an exact match; we treat ≥ 0.8 as acceptable. For finer-grained per-word feedback, the deployed `syllable-pattern-analyzer` skill computes a weighted composite of three sub-scores (position, length, and total similarity); see Appendix B.3 for the exact form. Equation 2 is the simplified core that all three sub-scores reduce to when patterns share equal length and total syllables. For example, "Let it go" has pattern $[1, 1, 1]$. A Chinese translation tokenized as two words with pattern $[2, 1]$ yields $\text{sim}([1, 1, 1], [2, 1, 0]) = 0.5$ (poor alignment), whereas a translation tokenized as three single-character words gives pattern $[1, 1, 1]$ with $\text{sim} = 1.0$.

4 IPA-Augmented Skill Framework

Figure 1 overviews the skill orchestration architecture.

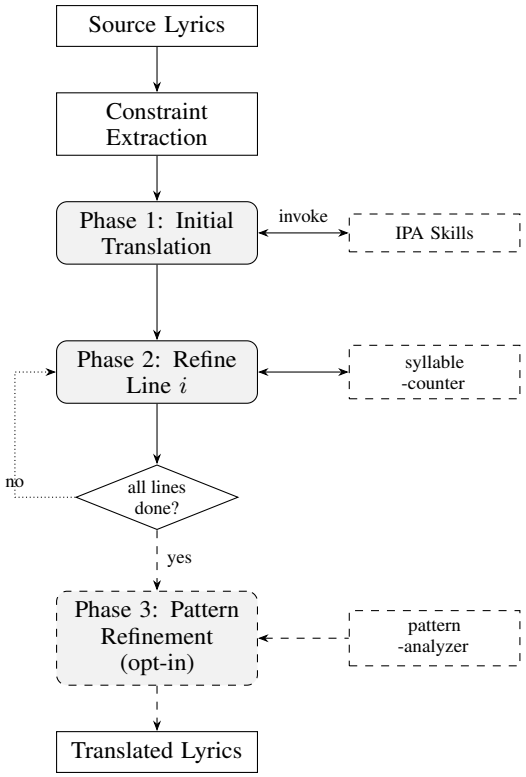

Figure 1: Skill framework overview. Phase 1 translates all lines using an internal tool-use loop that invokes the IPA skills. Phase 2 refines each line sequentially (up to 10 attempts per line); the dotted arrow shows the orchestration loop that iterates through lines. The dashed Phase 3 pass is an opt-in pattern-refinement stage; it is not part of the default pipeline (see Appendix E).

4.1 IPA as Universal Phonetic Interface

The International Phonetic Alphabet provides a standardized, language-independent representation of speech sounds, making it an ideal intermediary for cross-lingual phonetic analysis: regardless of the source or target language, all phonetic operations (syllable counting, rhyme comparison, pattern analysis) can be performed on IPA transcriptions using the same algorithms. Our system uses Phonemizer (Bernard and Titeux, 2021) as the text-to-IPA backend, wrapping the eSpeak NG speech synthesizer with support for over 100 languages. Language codes are normalized internally (e.g., `zh`, `zh-cn`, `cmn` all map to Mandarin Chinese) to provide a uniform interface. For phonetic distance computation, we use PanPhon (Mortensen et al., 2016), which maps each IPA segment to a vector of articulatory features, enabling fine-grained similarity measurement.

Skill	Description
`syllable-counter`	Count syllables via IPA vowel nuclei (or character count for Chinese)
`phonetic-analyzer`	Convert text to IPA via eSpeak NG and compute phonetic similarity
`rhyme-analyzer`	Detect the rhyme scheme by comparing IPA endings
`syllable-pattern-analyzer`	Compare target vs. current syllable patterns with structured feedback
`lyrics-validator`	Validate all three constraints jointly on a translated line set
`lyrics-translator`	Top-level orchestration skill coordinating the five sub-skills above through the multi-phase loop in §4.4

Table 1: Agent Skills exposed to the orchestrating LLM. Each skill is a directory with a `SKILL.md` description plus deterministic verification scripts; all skills accept a language parameter for language-specific processing.

4.2 Skill Suite

The orchestrator has access to a suite of Agent Skills (Table 1), each packaged as a `SKILL.md` description plus a deterministic verification script. The four IPA-based analytic skills (`syllable-counter`, `phonetic-analyzer`, `rhyme-analyzer`, `syllable-pattern-analyzer`) wrap functions that provide capabilities the LLM cannot reliably perform through text generation alone (Gao et al., 2023); `lyrics-validator` and `lyrics-translator` sit on top to compose them.

The `syllable-counter` skill is the most frequently invoked: given a text string and language code, it converts to IPA and returns an integer syllable count (§3.1). The `phonetic-analyzer` skill exposes the raw IPA transcription, allowing the orchestrator to inspect phonetic realizations and reason about which syllables to modify, a form of chain-of-thought reasoning (Wei et al., 2022) grounded in phonetic observation. The `rhyme-analyzer` skill analyzes the IPA endings of a set of lines and returns the rhyme scheme (e.g., "$AABB$"), ensuring judgments are phonetic rather than orthographic. The `syllable-pattern-analyzer` skill compares a target syllable pattern against the current one, returning the similarity score (Equation 2), per-word differences, and natural-language suggestions for improvement. All skills are discovered by the orchestrating LLM through their `SKILL.md` descrip-

Line	Text	Syll.	Pattern
1	The snow glows white	4	[1,1,1,1]
2	On the mountain tonight	6	[1,1,2,2]
3	Not a footprint to be seen	7	[1,1,2,1,1,1]
4	A kingdom of isolation	8	[1,2,1,4]
Rhyme scheme: $AABC$ (*white/tonight* rhyme via shared IPA ending /art/; lines 3–4 are unmatched)			

Table 2: Example constraint extraction from English source lyrics. Syllable counts are computed via IPA vowel nuclei; patterns via space-based word segmentation.

tions and invoked via the host's tool-use interface.

4.3 Constraint Extraction

Before translation begins, the system extracts all three constraints from the source lyrics L_s in language λ_s: (1) **syllable counts** $\mathbf{s} = (s_1, \ldots, s_n)$ via `syllable-counter` on each line, (2) **rhyme scheme** R_s via `rhyme-analyzer` on all lines, and (3) **syllable patterns** $P_s = (\mathbf{p}_1, \ldots, \mathbf{p}_n)$ via `syllable-pattern-analyzer`. These constraints are passed to the orchestrator as part of the translation prompt, providing explicit targets for each line. Table 2 shows an example.

4.4 Multi-Phase Skill Orchestration

Translation proceeds through two phases, encoded as procedural instructions in the top-level `lyrics-translator` `SKILL.md` that the orchestrating LLM follows, invoking the sub-skills at each step. The design reflects the priority ordering identified by Low (2005): semantic content and rhythm (syllable counts) take precedence, with rhyme considered throughout but not given a dedicated rewrite phase.

Phase 1: Initial Translation with Skill-Guided Reasoning. The orchestrator receives the source lyrics, target language, and all extracted constraints from the `lyrics-translator` skill instructions. It generates an initial translation of all lines, interleaving reasoning, skill invocations to verify output, and adjustments within the same generation turn (a tool-use pattern in the spirit of Yao et al. (2023)). The skill instructions direct the orchestrator to consider all three constraints simultaneously, producing a "best-effort" initial translation.

A typical interaction proceeds as: the orchestrator reasons about the target syllable count for a line, drafts a translation, invokes `syllable-counter` to verify, observes the result, and adjusts if needed, continuing until all lines are translated.

Phase 2: Line-by-Line Syllable Refinement. The initial translation rarely satisfies all syllable constraints exactly. In this phase, the orchestrator processes each line individually, invoking `syllable-counter` and comparing the result to the target. If there is a mismatch, the skill returns explicit feedback (e.g., "Line i has s_i' syllables but the target is s_i; please revise while preserving meaning") and the orchestrator generates a revised translation. The skill is re-invoked, continuing for up to 10 iterations; if no exact match is found, the closest attempt is retained. This phase is critical because even a single syllable mismatch can make a line unsingable.

Priority, rhyme handling, and termination. The two-phase design prioritizes syllable counts (the hardest constraint), then exits. Rhyme is monitored throughout via `rhyme-analyzer` and `lyrics-validator` but receives no dedicated refinement phase: rhyme violations are generally less disruptive to singability than syllable mismatches (Low, 2005), and our ablation (§6, Table 3) shows that a third syllable-pattern refinement phase actually degrades ARI by introducing rhyme structure absent from the source (Appendix E reports the design, prompt, and over-rhyming finding in full). We therefore adopt Phase 1+2 as the framework and treat phase 3 as an opt-in ablation studied separately. The process terminates after Phase 2 with a final `lyrics-validator` pass.

4.5 Worked Example

To illustrate, consider translating "*Not a footprint to be seen*" (7 syllables, pattern $[1, 1, 2, 1, 1, 1]$) into Chinese. In Phase 1, the orchestrator produces a 7-character translation; `syllable-counter` confirms the match, so Phase 2 skips this line. For lines that come out off-count, Phase 2 iterates up to 10 times with explicit count feedback until `syllable-counter` reports a match or the closest candidate is retained.

5 Evaluation Metrics

Standard MT metrics such as BLEU measure n-gram overlap and do not capture whether a translation preserves musical structure. Kim et al. (2023) proposed a computational evaluation framework for singable lyric translation; building on this direction, we propose three complementary metrics, each targeting one of the three musical constraints.

5.1 Syllable Error Rate (SER)

SER measures the overall alignment between the source and translated syllable count sequences using the Levenshtein edit distance (Levenshtein, 1966):

$$\text{SER} = \frac{\text{Lev}(\mathbf{s}, \mathbf{s}')}{\max(|\mathbf{s}|, |\mathbf{s}'|)} \tag{3}$$

where $\mathbf{s} = (s_1, \ldots, s_n)$ and $\mathbf{s}' = (s'_1, \ldots, s'_m)$ are the source and translated syllable count sequences, and Lev operates over sequences of integers (with substitution cost equal to 1 regardless of the magnitude of the difference).

SER captures both *substitution errors* (a line has the wrong syllable count) and *structural errors* (lines are missing or added). SER $= 0$ indicates perfect preservation; higher values indicate greater divergence. Unlike per-line metrics, SER penalizes translations that drop or add lines, which is important because line structure directly corresponds to melodic phrases.

5.2 Syllable Count Relative Error (SCRE)

SCRE provides a normalized, per-line view of syllable accuracy, assuming the translation preserves the number of lines ($m = n$):

$$\text{SCRE} = \frac{1}{n} \sum_{i=1}^{n} \frac{|s_i - s'_i|}{s_i} \tag{4}$$

SCRE weights errors relative to line length: a two-syllable error on a four-syllable line ($\text{SCRE}_i = 0.5$) is penalized more heavily than on a twelve-syllable line ($\text{SCRE}_i = 0.17$). This reflects the intuition that short lines leave less room for syllabic deviation; a single extra syllable in a three-syllable phrase is far more noticeable than in a long verse. SCRE $= 0$ indicates all lines match exactly.

SER and SCRE are complementary: SER captures structural alignment including line count mismatches, while SCRE provides percentage-based accuracy that is more interpretable for line-by-line comparison.

5.3 Adjusted Rand Index (ARI)

To measure rhyme scheme preservation, we treat the rhyme scheme as a *clustering* of lines and compute the Adjusted Rand Index (Hubert and Arabie, 1985) between the source and translated clusterings.

Let $R_s = (r_1, \ldots, r_n)$ and $R_t = (r'_1, \ldots, r'_n)$ be the rhyme scheme labelings. The Rand Index (RI) counts the fraction of line pairs that are either in the same cluster in both schemes or in different clusters in both schemes. ARI adjusts RI for the expected value under random label assignments:

$$\text{ARI} = \frac{\text{RI} - \mathbb{E}[\text{RI}]}{\max(\text{RI}) - \mathbb{E}[\text{RI}]} \tag{5}$$

ARI $\in [-1, 1]$: 1 indicates the translation perfectly preserves the rhyme structure, 0 indicates chance-level agreement, and negative values indicate systematic disagreement. Crucially, ARI is robust to label permutation: a source with scheme $AABB$ and a translation with $BBAA$ receives ARI $= 1$, since the grouping structure is identical.

Together, SER, SCRE, and ARI cover the structural dimensions of singable translation. They do not measure semantic fidelity or naturalness and are complementary to standard MT metrics such as BLEU and COMET.

6 Experiments

6.1 Setup

We evaluate on English-to-Chinese (en $\rightarrow$ cmn) song translation using 100 test cases (5 lines each) from the publicly available test split of the parallel lyric corpus released by Ou et al. (2023a).[2] The dataset covers pop, ballad, rock, and musical-theatre genres. Our open-source release (Apache-2.0) ships the full Skill suite (six `SKILL.md` directories with their verification scripts), the IPA tool implementations, and evaluation scripts under the `blt-skills` repository, enabling reproduction on any user-supplied lyrics in any language pair supported by eSpeak-NG.

Implementation. The framework is implemented as Agent Skills: each phase is encoded as procedural instructions in the orchestrating `lyrics-translator` `SKILL.md`, which references the five sub-skills (§4.2) and invokes their verification scripts as needed. For LLM inference, we evaluate Qwen3-30B (4-bit quantized) served locally via Ollama as a representative open-weight orchestrator. The same Skill suite is directly compatible with frontier models that support the Agent Skills interface (e.g., Claude via Claude Code), and supplementary Claude-orchestrated runs are reported in our open-source benchmark scripts.[3] The IPA

[2] https://huggingface.co/datasets/LongshenOu/lyric-trans-en2zh-data

[3] See `scripts/run_cc.py` and `scripts/run_bench.py` in the released code.

Method	SER ↓	SCRE ↓	ARI ↑
Phase 1 only	0.8343	0.2076	0.3133
Phase 1+2	<u>0.2151</u>	<u>0.0291</u>	**0.3623**
Phase 1+2+3 (full)	0.3132	0.0457	0.2951
CLT (Ou et al., 2023a)	**0.0860**	**0.0142**	0.0015

Table 3: Phase ablation and baseline comparison (en → cmn, n=100). Bold = best overall; underline = best among BLT variants.

verification scripts are exposed to the orchestrator through each skill's `SKILL.md` description.

Conditions. We conduct a phase ablation study alongside a supervised baseline:

- **Phase 1 only**: Initial translation with IPA skill access but no iterative refinement.
- **Phase 1+2**: Adds line-by-line syllable count refinement (up to 10 iterations per line).
- **Phase 1+2+3** (full BLT Skills): Adds syllable pattern refinement after syllable counts are matched.
- **CLT** (Ou et al., 2023a): A supervised mBART model fine-tuned with explicit length, rhyme, and word-boundary constraint tokens. This represents a task-specific training approach and serves as an upper bound for constraint satisfaction.

All BLT conditions use the same Qwen3-30B orchestrator and the same Skill suite; CLT uses its own fine-tuned checkpoint. We report SER, SCRE, and ARI as defined in §5.

6.2 Results and Phase Ablation

Table 3 presents the results of the phase ablation study.

Phase 2 is the critical stage. Adding syllable refinement reduces SER by 74.2% (0.83→0.22) and SCRE by 86.0% (0.21→0.03), confirming that iterative skill-verified refinement, not merely skill access during initial generation, drives constraint satisfaction. The cost is a 4.8× increase in inference time (7.0 s → 33.5 s per test case on a single NVIDIA RTX 3090; timings include all phases up to and including the indicated phase). This latency remains practical for offline lyric translation, where constraint satisfaction outweighs speed.

Phase 3 over-rhymes and does not help. An optional Phase 3 pattern-refinement pass (full description in Appendix E) actively hurts every metric on this benchmark: SER and SCRE rise by 45.6% and 57.0% as adjusting word boundaries disrupts syllable counts Phase 2 had matched, and ARI drops by 18.6% (0.36→0.30) because the rewrite tends to introduce same-final endings absent from the source. We therefore adopt **Phase 1+2 as the default** and do not run Phase 3 unless explicitly requested.

Comparison with CLT. The supervised CLT baseline achieves strong syllable accuracy (SCRE 0.014) through explicit constraint tokens and task-specific fine-tuning. However, its ARI is near zero (0.002), indicating that its constraint mechanism does not preserve rhyme structure. We stress that this ARI gap is not an artifact of permissive rhyme detection: the scheme builder used to compute ARI requires *exact* ending equality (Appendix B.2, step 2), the same strict criterion applied to both BLT and CLT outputs, so the near-zero ARI for CLT reflects a genuine lack of inter-line rhyme structure (e.g., $AABB$ vs. $ABAB$) rather than a thresholding artifact. The best BLT variant (Phase 1+2) achieves substantially higher ARI (0.36 vs. 0.002) while requiring no task-specific training, only IPA skills and a general-purpose LLM. CLT is also limited to the single language pair it was trained on, whereas BLT operates on any language pair supported by Phonemizer's eSpeak-NG backend, covering over 100 languages without retraining.

6.3 Multi-Orchestrator Ablation

We replicate the pipeline with three Claude orchestrators (Haiku 4.5, Sonnet 4.6, Opus 4.7) on the Ou 2023 en→zh test split (seed=42, n=100), under a *vanilla* single-prompt condition and the full Phase 1+2+3 pipeline used for the Qwen3-30B run in Table 3; we include Phase 3 here so each orchestrator runs the same pipeline as Qwen3-30B and to surface the same over-rhyming behaviour identified in §6 (Appendix E). Each Claude call uses `-effort medium` via the Claude Code CLI; other hyperparameters match Appendix C.

Findings. Vanilla Claude's SER (0.86–0.88) is indistinguishable from Qwen3-30B Phase 1's 0.83 (Table 3), confirming that without the IPA skill suite even a frontier LLM cannot honor syllable counts. Under Phase 1+2+3, Haiku and Opus achieve SER=SCRE=0 on every case, with Sonnet at a single 4/5 near-miss (SER 0.002). ARI rises

Orchestrator	SER ↓	SCRE ↓	ARI ↑	LLM	Time
Vanilla (no skills, single prompt; n=100)					
Haiku 4.5	0.872	0.350	0.196	1.0	11.0
Sonnet 4.6	0.860	0.277	0.171	1.0	5.9
Opus 4.7	0.880	0.303	0.185	1.0	6.3
Full pipeline (Phase 1+2+3)					
Qwen3-30B	0.313	0.046	0.295	–	33.5
Haiku 4.5	**0.000**	**0.000**	0.509	5.5	501.7
Sonnet 4.6	0.002	0.001	0.653	5.2	180.9
Opus 4.7	**0.000**	**0.000**	**0.693**	5.3	94.6

Table 4: Multi-orchestrator ablation on Ou 2023 en→zh test split (seed=42, n=100 for all rows). **LLM**: mean LLM calls per case across all phases (Vanilla = 1 call by construction). **Time**: mean wall-clock seconds per case. Qwen3-30B numbers reproduced from Table 3.

Line	Text	Syll.	Tgt
Source (English):			
1	The snow glows white	4	–
2	On the mountain tonight	6	–
3	Not a footprint to be seen	7	–
4	A kingdom of isolation	8	–
Base LLM (Chinese, no tools):			
1	白雪在夜裡閃耀	7	4 ×
2	今夜在那座山上	7	6 ×
3	看不見任何一個腳印	9	7 ×
4	一個與世隔絕的王國	9	8 ×
BLT Skills (Chinese, with IPA skills):			
1	皚皚白雪	4	4 ✓
2	籠罩今夜山巔	6	6 ✓
3	看不到一絲足跡	7	7 ✓
4	這是孤獨的國度	7	8 ×

Table 5: Qualitative example: English-to-Chinese translation of the opening lines of *Let It Go*. "Syll." is the actual character count; "Tgt" is the source syllable count that the translation should match. BLT Skills matches 3/4 targets; the Base LLM matches 0/4.

sharply with orchestrator capability (Opus 0.693, Sonnet 0.653, Haiku 0.509), all far above Qwen3-30B's 0.295 and CLT's 0.002. Wall-clock latency follows the inverse ordering (Opus 94.6s, Sonnet 180.9s, Haiku 501.7s).

6.4 Qualitative Analysis

Table 5 illustrates the pipeline on the *Let It Go* excerpt from Table 2. The Base LLM overshoots every target (7–9 characters), whereas BLT Skills, guided by `syllable-counter` verification in Phase 2, matches three of four targets exactly; Line 4 (target 8) remains one syllable short at 7 after Phase 2 exhausted its 10 refinement attempts.

7 Conclusion

We presented a framework for lyrics translation that preserves musical constraints through Agent Skills, formalizing three constraints (syllable counts, rhyme schemes, syllable patterns) with IPA-based extraction and verification, and structuring translation as a two-phase orchestration over composable Skills that prioritizes syllable counts while monitoring rhyme throughout. The proposed SER, SCRE, and ARI metrics complement standard translation quality evaluation.

A phase ablation on English-to-Chinese (n=100) shows iterative syllable refinement (Phase 2) drives the majority of improvement (86.0% SCRE reduction), and an additional pattern-refinement pass (Appendix E) over-rhymes and is left opt-in; we therefore adopt Phase 1+2 as the default. Compared with a supervised baseline (Ou et al., 2023a), BLT preserves rhyme structure (ARI 0.36 vs. 0.002) without task-specific training and, unlike constrained decoding (Hokamp and Liu, 2017; Post and Vilar, 2018), remains language-agnostic; future work targets the accuracy gap through stronger LLMs and integration with singing voice synthesis (Ren et al., 2020).

Limitations

Our approach has several limitations. First, the framework's correctness depends on Phonemizer / eSpeak-NG for IPA transcription, which has uneven quality across languages; we do not provide an intrinsic evaluation of syllable-counting or rhyme-detection accuracy. Second, the framework lacks a dedicated rhyme refinement phase; our opt-in Phase 3 pattern-refinement pass (Appendix E) over-rhymes rather than improves rhyme alignment, so we leave it out of the default pipeline. Third, our evaluation covers only English-to-Chinese (n=100) with a single LLM; although the framework is language-agnostic in principle, we have not validated it on other language pairs. Fourth, the supervised CLT baseline substantially outperforms BLT on syllable metrics, indicating that the agent approach has room to improve on constraint satisfaction; the current advantage is in rhyme preservation and language-pair flexibility rather than raw accuracy. Fifth, the multi-phase refinement loop introduces latency (33.5 s for Qwen3-30B Phase 1+2+3 on a single RTX 3090 and 94.6–501.7 s for the Claude orchestrators via the API, vs. 1.4 s for CLT

on Apple Silicon MPS per test case), which may limit practical deployment. Sixth, our metrics measure structural constraint preservation but not semantic fidelity or naturalness; a complete evaluation requires human judgments. Finally, the technical approach of invoking deterministic phonetic skills from an orchestrating LLM applies established patterns (Yao et al., 2023; Gao et al., 2023; Schick et al., 2023) to a new domain; the primary contribution is the formalization of musical constraints and the IPA-based Skill suite rather than a novel agent architecture.

Ethical considerations. Song lyrics are protected by copyright in most jurisdictions. Our quantitative evaluation uses the en→zh test split released by Ou et al. (2023a), redistributed under the terms of that release; the qualitative *Let It Go* excerpts in Tables 2 and 5 are reproduced as short fragments for academic illustration under fair-use principles. Our open-source code does not bundle any lyric corpora; users supply their own input lyrics and are responsible for ensuring they hold the rights to translate and redistribute them. The framework's reliance on Phonemizer/eSpeak-NG also inherits that backend's uneven coverage across languages: dialectal, low-resource, or non-standard varieties may receive lower-quality IPA, which would propagate into both syllable counts and rhyme detection; users translating into such varieties should treat the structural metrics as advisory rather than authoritative. As with any high-fidelity translation tool, BLT Skills could in principle be used to produce singable derivative versions without rights-holder consent; we view rights-clearance as the user's responsibility and recommend that downstream services pair the framework with provenance tracking or licensing checks.

References

Mathieu Bernard and Hadrien Titeux. 2021. Phonemizer: Text to phones transcription for multiple languages in Python. *Journal of Open Source Software*, 6(68):3958.

Jiale Chen, Xuelian Dong, Qihao Yang, Wenxiu Xie, and Tianyong Hao. 2025. Can large language models translate spoken-only languages through international phonetic transcription? In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 23431–23446.

Yiwen Chen and Simone Teufel. 2024. Scansion-based lyrics generation. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 14370–14381.

Woohyun Cho, Youngmin Kim, Sunghyun Lee, and Youngjae Yu. 2025. MAVL: A multilingual audio-video lyrics dataset for animated song translation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.

Johan Franzon. 2008. Choices in song translation: Singability in print, subtitles and sung performance. *The Translator*, 14(2):373–399.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, pages 10764–10799.

Marjan Ghazvininejad, Yejin Choi, and Kevin Knight. 2018. Neural poetry translation. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*.

Marjan Ghazvininejad, Xing Shi, Yejin Choi, and Kevin Knight. 2016. Generating topical poetry. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1183–1191.

Fenfei Guo, Chen Zhang, Zhirui Zhang, Qixin He, Kecheng Zhang, Jun Xie, and Jordan Boyd-Graber. 2022. Automatic song translation for tonal languages. In *Findings of the Association for Computational Linguistics: ACL 2022*.

Chris Hokamp and Qun Liu. 2017. Lexically constrained decoding for sequence generation using grid beam search. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1535–1546.

Jack Hopkins and Douwe Kiela. 2017. Automatically generating rhythmic verse with neural networks. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 168–178.

Lawrence Hubert and Phipps Arabie. 1985. Comparing partitions. *Journal of Classification*, 2(1):193–218.

Wenxiang Jiao, Wenxuan Wang, Jen-tse Huang, Xing Wang, Shuming Shi, and Zhaopeng Tu. 2023. Is ChatGPT a good translator? Yes with GPT-4 as the engine. *arXiv preprint arXiv:2301.08745*.

Haven Kim, Jongmin Jung, Dasaem Jeong, and Juhan Nam. 2024. K-pop lyric translation: Dataset, analysis, and neural-modelling. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 9974–9987.

Haven Kim, Kento Watanabe, Masataka Goto, and Juhan Nam. 2023. A computational evaluation framework for singable lyric translation. In *Proceedings of the 24th International Society for Music Information Retrieval Conference*.

Jey Han Lau, Trevor Cohn, Timothy Baldwin, Julian Brooke, and Adam Hammond. 2018. Deep-speare: A joint neural model of poetic language, meter and rhyme. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1948–1958.

Vladimir I. Levenshtein. 1966. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710.

Qihao Liang, Xichu Ma, Finale Doshi-Velez, Brian Lim, and Ye Wang. 2024. XAI-lyricist: Improving the singability of AI-generated lyrics with prosody explanations. In *Proceedings of the 33rd International Joint Conference on Artificial Intelligence*, pages 7877–7885.

Peter Low. 2005. The pentathlon approach to translating songs. In Dinda L. Gorlée, editor, *Song and Significance: Virtues and Vices of Vocal Translation*, pages 185–212. Rodopi, Amsterdam.

Peter Low. 2022. Translating the texts of songs and other vocal music. In Kirsten Malmkjær, editor, *The Cambridge Handbook of Translation*. Cambridge University Press.

David R. Mortensen, Patrick Littell, Akash Bharadwaj, Kartik Goyal, Chris Dyer, and Lori Levin. 2016. PanPhon: A resource for mapping IPA segments to articulatory feature vectors. In *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*, pages 3475–3484.

Longshen Ou, Xichu Ma, Min-Yen Kan, and Ye Wang. 2023a. Songs across borders: Singable and controllable neural lyric translation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 447–467.

Longshen Ou, Xichu Ma, and Ye Wang. 2023b. LOAF-M2L: Joint learning of wording and formatting for singable melody-to-lyric generation. *arXiv preprint arXiv:2307.02146*.

Matt Post and David Vilar. 2018. Fast lexically constrained decoding with dynamic beam allocation for neural machine translation. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 1314–1324.

Le Ren, Xiangjian Zeng, Qingqiang Wu, and Ruoxuan Liang. 2025. LyriCAR: A difficulty-aware curriculum reinforcement learning framework for controllable lyric translation. In *arXiv preprint arXiv:2510.19967*.

Yi Ren, Xu Tan, Tao Qin, Jian Luan, Zhou Zhao, and Tie-Yan Liu. 2020. DeepSinger: Singing voice synthesis with data mined from the web. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1979–1989.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the Eleventh International Conference on Learning Representations*.

Zhuorui Ye, Jinhan Li, and Rongwu Xu. 2024. Sing it, narrate it: Quality musical lyrics translation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*.

Suhyeon Yoo, Khai N. Truong, and Young-Ho Kim. 2025. ELMI: Interactive and intelligent sign language translation of lyrics for song signing. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*.

Jiaxing Yu, Xinda Wu, Yunfei Xu, Tieyao Zhang, Songruoyao Wu, Le Ma, and Kejun Zhang. 2025. SongGLM: Lyric-to-melody generation with 2D alignment encoding and multi-task pre-training. In *Proceedings of the 39th AAAI Conference on Artificial Intelligence*.

Songyan Zhao, Bingxuan Li, Yufei Tian, and Nanyun Peng. 2025. REFFLY: Melody-constrained lyrics editing model. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 11295–11315.

A Prompt Templates

We provide the full prompt templates used in each phase of the two-phase orchestration described in §4.4 (the opt-in Phase 3 prompt is reproduced in Appendix E). All prompts live inside `skills/lyrics-translator/SKILL.md` (and the phase-specific sub-skill descriptions); the orchestrating LLM loads them when the parent skill is invoked, and reads each sub-skill's `SKILL.md` when the corresponding tool is needed.

System prompt (all phases, from `lyrics-translator/SKILL.md`).

```
You are a lyrics translator. Translate
ONE line at a time while matching the
target syllable count.

Available skills:
- syllable-counter: count syllables of a
candidate translation
- lyrics-validator: verify a line
against all three constraints

WORKFLOW:
1. Translate the source line
2. Invoke syllable-counter to check your
translation
3. If the count does not match, revise
and re-check
4. Once correct, output the final
translation with no punctuation

IMPORTANT: Always verify with the skills
before finalizing your answer.
```

Phase 1: Initial translation. The orchestrator receives all source lines with their target syllable counts, the detected rhyme scheme, and the target syllable patterns. It is instructed to translate *all* lines simultaneously, invoking the relevant skills to verify constraints during generation:

```
Translate ALL N lines of the following
lyrics to {target_lang} while meeting
ALL 3 musical constraints.

CONSTRAINT 1: SYLLABLE COUNTS (MUST
MATCH EXACTLY)
CONSTRAINT 2: RHYME SCHEME:
{rhyme_scheme}
CONSTRAINT 3: SYLLABLE PATTERNS:
{patterns}

Invoke the available skills to verify:
(1) syllable counts match exactly, (2)
rhyme scheme is correct, (3) syllable
patterns are maintained.

Output exactly N translated lines,
numbered 1-N.
```

Phase 2: Syllable refinement. For each mismatched line, the orchestrator receives the current best translation and explicit corrective feedback from `syllable-counter`:

```
Adjust this translation to have EXACTLY
{target} syllables. Focus ONLY on
syllable count. Minimize meaning
changes.

Original: "{source_line}"
Current: "{best_translation}"
Current syllables: {actual}/{target}
Action: {add/remove k syllables}

STRATEGIES:
- If too long: remove adjectives, use
shorter words, merge concepts
- If too short: add descriptive words,
use longer characters

Output ONLY the adjusted translation.
```

B Skill Algorithms

We describe the core algorithms underlying each IPA skill.

B.1 `syllable-counter`

1. Remove punctuation from input text.
2. **Chinese:** Return the character count directly (each character = one syllable).
3. **Other languages:** Convert text to IPA via Phonemizer / eSpeak-NG.
4. Match all diphthong and monophthong nuclei against the inventories below; diphthongs are tried first so adjacent vowels forming a single nucleus are not double-counted.

   Diphthongs (longest-match, in order):

   aɪ, eɪ, ɔɪ, aʊ, oʊ, ɪə, eə, ʊə, aɪə, aʊə

   Monophthong nuclei: a fixed Unicode character class of 27 IPA vowel symbols spanning four articulatory families: cardinal vowels (close to open, front to back, e.g. i, ɛ, a, u), central vowels (e.g. ə, ɜ), rounded front vowels (e.g. y, ø, œ), and back-unrounded plus rhotacised vowels. The exact 27-codepoint inventory is enumerated in `src/blt_skills/phonetics.py:13-16` of the open-source release; the matcher uses only this set and never matches consonant symbols.

   The matcher tolerates trailing combining marks (length, nasalization, tone) on each nucleus. The inventory contains vowel symbols only; consonant symbols never appear in it.

5. Return the number of matches as the syllable count (one match per vowel nucleus).

We treat eSpeak-NG's IPA output as ground truth for vowel nuclei.

B.2 `rhyme-analyzer`

1. For each line, extract the *rhyme ending*:
   - **Chinese:** Use PyPinyin to extract the final (韻母) of the last character.
   - **Other languages:** Convert to IPA, find the last vowel nucleus, return the substring from that position to end-of-string.
2. Build the rhyme scheme used for ARI by iterating through lines and grouping by *exact* ending equality: maintain a hash map from ending strings to labels; for each line, if its ending key is already in the map, reuse the label, otherwise assign the next unused label and insert it.

3. Pair-level rhyme judgments inside the `lyrics-validator` skill use a more permissive rule: two lines rhyme if $\texttt{ending}_1 == \texttt{ending}_2 \vee \texttt{ending}_1 \subset \texttt{ending}_2 \vee \texttt{ending}_2 \subset \texttt{ending}_1$.

The substring rule in step 3 lets the validator accept a short ending like /aɪ/ as rhyming with a longer one like /aɪt/, mirroring the audible vowel match that drives perceived rhyme in singing. Crucially, the substring rule is *not* used when constructing the scheme that feeds into the ARI metric: ARI is computed on the strict, exact-match scheme produced in step 2, so reported ARI values are not inflated by the validator's permissiveness. A natural extension would replace exact equality in step 2 with a PanPhon (Mortensen et al., 2016) feature-distance threshold on the rhyming nuclei; we leave a sensitivity analysis of this threshold to future work.

B.3 `syllable-pattern-analyzer`

Given target pattern $\mathbf{p}$ and current pattern $\mathbf{p}'$:

1. Compute position-by-position differences for each word position j: $d_j = p'_j - p_j$ (zero-padded to the longer length).
2. Compute three sub-scores:
   - *Position similarity* (weight 0.5): $1 - \sum_j |d_j| / \sum_j p_j$
   - *Length similarity* (weight 0.25): $1 - ||\mathbf{p}| - |\mathbf{p}'|| / \max(|\mathbf{p}|, |\mathbf{p}'|)$
   - *Total similarity* (weight 0.25): $1 - |\sum p_j - \sum p'_j| / \sum p_j$
3. Return the weighted sum, clipped to $[0, 1]$, along with per-word suggestions (e.g., "word 3: has 3 syllables, target is 1").

Note: the similarity in the main paper (Equation 2) uses the simplified formulation; the implementation adds length and total sub-scores for finer-grained feedback.

C Hyperparameters

Table 6 lists all hyperparameters used in the experiments.

The pattern threshold of 0.8 was chosen empirically: values below 0.6 indicate poor alignment, 0.6–0.8 indicate minor mismatches, and $\geq$0.8 indicates acceptable alignment for singing. Phase 3 accepts a revised translation only if it strictly improves pattern similarity *and* maintains the correct total syllable count; otherwise the revision is discarded.

Category	Parameter	Value
Model	LLM	Qwen3-30B
Model	Quantization	Q4_K_M
Model	Temperature	0.7
Phase 1	Max skill invocations	10
Phase 1	Skill set	All 6 skills
Phase 2	Max attempts / line	10
Phase 2	Skill calls / attempt	5
Phase 3	Pattern threshold	0.8
Phase 3	Accept condition	$\text{sim}' > \text{sim}$
Orchestr.	Recursion limit	50
Orchestr.	Max lines / test	5
Segment.	English	Whitespace
Segment.	Chinese	HanLP

Table 6: Hyperparameters for all experiments.

D Extended Qualitative Examples

D.1 Success Case: Exact Constraint Match

Table 7 shows a test case where the BLT Skills system achieves perfect syllable counts on all lines. Phase 1 produces a close initial translation (4/5 lines correct); Phase 2 fixes the one remaining line in 2 iterations.

L	Source	Tgt	P1	P2
1	Let it go, let it go	6	6 ✓	6 ✓
2	Can't hold it back anymore	7	8 ×	7 ✓
3	Let it go, let it go	6	6 ✓	6 ✓
4	Turn away and slam the door	7	7 ✓	7 ✓
5	I don't care	3	3 ✓	3 ✓

Table 7: Success case (en → cmn). "P1" and "P2" show syllable counts after Phase 1 and Phase 2 respectively. Phase 2 corrects line 2 from 8 to 7 syllables.

D.2 Failure Case: Persistent Mismatch

Table 8 shows a case where Phase 2 cannot fully resolve a mismatch. Line 3 requires 9 syllables but the agent converges on 8 after exhausting 10 attempts, the closest semantically coherent translation it can find.

This oscillation pattern, where the agent alternates between undershooting and overshooting, accounts for many Phase 2 failures, particularly when the target count falls between two "natural" translation lengths.

L	Source	Tgt	P1	P2
1	The snow glows white	4	4 ✓	4 ✓
2	On the mountain tonight	6	6 ✓	6 ✓
3	Not a footprint to be seen	7	9 ×	7 ✓
4	A kingdom of isolation	8	9 ×	7 ×
5	And it looks like I'm the queen	8	8 ✓	8 ✓

Table 8: Failure case (en → cmn). Line 4 targets 8 syllables; Phase 2 reduces from 9 to 7 but overshoots, and subsequent attempts oscillate between 7 and 9 without hitting 8.

E Phase 3 Pattern Refinement

Phase 3 is an opt-in third stage that operates after Phase 2 has matched syllable counts. We describe its design, prompt, and the over-rhyming behaviour that motivates leaving it out of the default pipeline.

Design. After syllable counts are matched, the orchestrator refines word-level syllable distributions. For each line, `syllable-pattern-analyzer` compares the current pattern against the target and returns the similarity score (Eq. 2), per-word differences, and natural-language suggestions (e.g., "split the first word into shorter components"). For each line whose similarity is below the threshold (0.8 by default), the orchestrator generates a revised translation; the revision is accepted only if it strictly improves pattern similarity *and* preserves the total syllable count, otherwise it is reverted to the Phase 2 output.

Prompt template.

> `Adjust the word distribution in this translation without changing total syllable count.`
>
> `Current pattern: {current} (total: `s` syllables)`
> `Target pattern: {target} (total: `s` syllables)`
>
> `Adjustments needed: {per-word suggestions}`
>
> `Keep total syllable count at `s`. Adjust word choice to match the target pattern.`
>
> `Output ONLY the adjusted translation.`

Over-rhyming finding. On an apples-to-apples comparison of Phase 1+2 against Phase 1+2+3 over the same 50 cases of the Ou en→zh test split (Haiku 4.5, seed 42), Phase 3 degrades mean ARI from 0.691 to 0.538 (10 cases better, 22 same, 18 worse) while preserving SER and SCRE at zero, at 4.4× the LLM calls and 4.0× the wall-clock time. Inspecting per-case outputs, the regressions are not random sampling noise: the rewrites repeatedly introduce same-final line endings that are absent from the source, for example ending consecutive lines with sentence-final particles 吧//啊 or repeating a single pinyin final (e.g. 海 (ai) at lines 1 and 5 of a song whose source has no A . . . A pattern). Rhyme that does not match the source reduces ARI even though it sounds rhyming, which surfaces a limitation of the current Phase 3 prompt: it optimizes pattern similarity in isolation, with no signal about the source rhyme structure that the rewrite should preserve. A conservative fix, such as conditioning the rewrite on the source rhyme labels or requiring the rewrite to keep each line's pinyin final unchanged, is a natural direction for future work.

Illustrative example. Table 9 shows a case where Phase 3 instead *improves* rhyme alignment; such cases exist but are outweighed by regressions in aggregate. The source has scheme $AABB$; after Phase 2, the translation has $ABCD$ (no rhymes preserved); Phase 3's pattern adjustments happen to produce line endings that restore partial rhyme structure ($AABC$).

L	Source	Rhyme (P2)	Rhyme (P3)
1	silver *light*	A	A
2	purest *white*	B	A
3	world *below*	C	B
4	to *blow*	D	C
	Scheme:	$ABCD$	$AABC$
	ARI:	-0.20	0.33

Table 9: Phase 3 rhyme effect on one favourable case. Source scheme $AABB$; Phase 2 output has no rhyme structure ($ABCD$, ARI $= -0.20$); Phase 3 partially restores it ($AABC$, ARI $= 0.33$).

F CLT Baseline Details

The Controllable Lyric Translation (CLT) baseline (Ou et al., 2023a) uses a fine-tuned mBART model with explicit constraint tokens prepended to the encoder input. We use the publicly released checkpoint `LongshenOu/lyric-trans-en2zh`.

Constraint encoding. CLT encodes three constraints as special tokens: (1) a *length token* specifying the target character count, (2) a *rhyme token* specifying the target rhyme class (e.g., `a/ia/ua`), and (3) a *word boundary token* vector indicating which positions should be word boundaries. The model generates characters in *reverse order* (right-to-left) and the output is flipped to produce the final translation.

Reported metrics. The original paper reports 99.85% length accuracy and 99.00% rhyme accuracy on their test set. Our evaluation on the same test split yields SER $= 0.086$ and SCRE $= 0.014$, consistent with strong length control. However, ARI $= 0.002$, indicating that CLT's rhyme tokens enforce line-level rhyme category (e.g., "the last character should rhyme with *a*") but do not preserve the *inter-line rhyme scheme structure* (e.g., $AABB$ vs. $ABAB$).

Inference time. On a 30-case sample of the en→zh test split, CLT averages 1.4 s per case (median 1.5 s, range 0.6–1.9 s) on Apple Silicon MPS with `num_beams=5` and `max_length=36`. This places CLT roughly an order of magnitude faster than Qwen3-30B Phase 1+2+3 (33.5 s on RTX 3090; Table 3) and two orders of magnitude faster than the Claude orchestrators (94.6–501.7 s via API; Table 4). The gap is intrinsic to the architectures: CLT is a 600M-parameter mBART decoded once per song with constrained beam search, whereas the BLT pipeline issues multiple LLM calls per line (mean 5.2–5.5 calls per case). Hardware comparison is therefore approximate; even on identical hardware CLT would remain substantially faster.

Key differences from BLT. CLT requires: (1) a parallel lyric corpus for fine-tuning, (2) per-language-pair training, and (3) pre-specified constraint values at inference time. BLT requires none of these: it extracts constraints automatically from source lyrics and operates with any general-purpose LLM. The trade-off is that CLT achieves much higher syllable accuracy while BLT provides better rhyme preservation and language flexibility.